

IPL Win-Probability Models - Held-Out Evaluation Guide

Part 1: How the evaluation was performed (repeatable steps) • Part 2: Likely interview questions with model answers. All numbers were actually computed from the project's ball-by-ball dataset and are byte-for-byte reproducible.

Results reference (held-out season 2024, 8,185 second-innings states)

Model	Accuracy	ROC-AUC	Precision	Recall
XGBoost	0.8137	0.8977	0.8651	0.7209
Random Forest	0.8077	0.8935	0.8516	0.7217
Logistic Regression	0.7796	0.8587	0.8196	0.6883

Part 1 - Repeatable Methodology

- **Step 1 - Explore the repository first**

Locate three things before writing any evaluation code: the raw data (data/raw/matches.csv - 1 row per match; data/raw/deliveries.csv - 1 row per ball), the training code (src/ipl_analytics/models.py, scripts/train_models.py) and any saved artifacts (models/*.joblib). The training code is the source of truth for features, hyperparameters and target.

- **Step 2 - Identify the correct models and target column**

The project has two classifier families. The win-probability models are live_random_forest / live_xgboost / live_logistic_regression trained by train_live_chase_classifier on build_match_context_dataset output: one row per ball of every second innings (125,393 rows). Target column 'won' is already int 0/1 (1 = chasing team wins), so no label mapping is needed.

- **Step 3 - Choose a temporal (season-based) split, not a random one**

Rows are time-ordered; a random shuffle lets future matches influence training and inflates metrics. Use: test_season = int(context['season'].max()) = 2024; train = seasons < 2024 (117,208 rows); test = season == 2024 (8,185 rows). Persist the test set to data/test_set.csv for auditing.

- **Step 4 - Do NOT evaluate the saved artifacts directly - retrain instead**

The saved .joblib models were fitted on ALL seasons including 2024 (random split inside _classify_score). Scoring them on season 2024 means many test rows were seen during training = leakage. Fix: rebuild pipelines with identical recipes and seeds, fit only on seasons < 2024. This mirrors the repo's own _chrono_evaluate_classifier pattern.

- **Step 5 - Replicate preprocessing exactly**

From _preprocessor in src/ipl_analytics/models.py: categorical ['batting_team','bowling_team','venue'] -> SimpleImputer(strategy='most_frequent') -> OneHotEncoder(handle_unknown='ignore'); numeric (target, current_score, runs_needed, overs_remaining, balls_remaining, wickets_remaining, current_run_rate, required_run_rate, run_rate_gap, balls_per_wicket, pace_ratio) -> SimpleImputer(strategy='median'); StandardScaler ONLY for Logistic Regression (matches original pipeline; trees are scale-invariant).

- **Step 6 - Pin reproducibility controls**

np.random.seed(42); random.seed(42); every estimator gets random_state=42 and n_jobs=1. XGBoost uses eval_metric='logloss'. (use_label_encoder=False is obsolete in xgboost >= 2.0.) Classification threshold fixed at 0.5 everywhere, matching repo code.

- **Step 7 - Fit, predict, score**

pipeline.fit(X_train, y_train); y_proba = pipeline.predict_proba(X_test)[: , 1]; y_pred = (y_proba >= 0.5).astype(int); then accuracy_score(y_test, y_pred), roc_auc_score(y_test, y_proba), precision_score(y_test, y_pred, zero_division=0), recall_score(y_test, y_pred, zero_division=0). Round to 4 decimals into evaluation_results.csv.

- **Step 8 - Persist evidence and verify determinism**

Write evaluation_steps.txt (exact commands, calls, hyperparameters), environment.txt (python/pip freeze versions).
Re-run the whole pipeline a second time and compare SHA-256 hashes of the outputs - they matched byte-for-byte.

Exact reproduction

```
# from project root
python scripts/run_full_evaluation.py

# key files produced
evaluation_results.csv # model, accuracy, roc_auc, precision, recall
evaluation_steps.txt   # full audit log of every call made
environment.txt        # pinned package versions
data/test_set.csv      # held-out season 2024 rows
scripts/preprocess_test.py # feature lists + pipeline builder
scripts/run_full_evaluation.py # end-to-end orchestrator
```

Part 2 - Interview Questions & Answers

Every question below maps to a real decision made in this evaluation. Answering these fluently makes the resume numbers fully defensible.

Q: Why did you use a season-based split instead of train_test_split?

A: The rows are time-ordered sports events. A random stratified split would place balls from the same match - even the same innings - on both sides of the split, so the model effectively memorises outcomes it should predict. Holding out the most recent season (2024) simulates real deployment: train on history, predict matches that happen later. It also exposes concept drift (rule changes, venue shifts, player turnover between eras).

Q: Your saved models existed - why did you retrain them?

A: The artifacts had been fitted on the full dataset, which includes the held-out season. Evaluating them there would leak training rows into the test set and overstate performance. I rebuilt the pipelines with identical hyperparameters and seeds but fitted only on seasons < 2024 - exactly the refit-on-prior-data pattern the repo itself uses in `_chrono_evaluate_classifier`.

Q: ROC-AUC ~0.90 seems very high for cricket. Is something wrong?

A: No - and it's important to explain why. This is a LIVE win-probability task: at over 18 with 20 needed off 12, the outcome is largely determined by the state itself, so probability estimates naturally sharpen as the chase progresses. Suspicious signals would be near-perfect accuracy or identical train/test scores; instead we see realistic error rates (accuracy ~0.81) and a drop versus in-sample metrics. The pre-match winner models in the same repo sit near 0.50 AUC, which shows the pipeline isn't leaking.

Q: Interpret precision and recall for this model.

A: Precision 0.865: when XGBoost says the chasing team wins, it's right ~87% of the time - relevant if you act on positive predictions. Recall 0.721: it catches ~72% of all actual chase wins. The gap comes from the class mix and hard late-swing states; you'd tune the 0.5 threshold depending on whether false positives or false negatives cost more.

Q: Why OneHotEncoder(handle_unknown='ignore')?

A: Season 2024 can contain venues or team pairings never seen in training. With `handle_unknown='error'` the transform would crash at inference; 'ignore' encodes an unseen category as all zeros, letting the numeric match-state features carry the prediction. Median imputation keeps occasional missing numerics from breaking the batch.

Q: Why scale features only for Logistic Regression?

A: That mirrors the original training pipeline exactly. Tree ensembles split on thresholds and are invariant to monotone scaling; logistic regression benefits from standardised features for stable convergence and fair regularisation. Reproducing preprocessing faithfully matters more than debating the choice - otherwise you're evaluating a different model than the repo ships.

Q: How did you guarantee reproducibility?

A: Seeds fixed for Python random, NumPy and each estimator (`random_state=42`), `n_jobs=1` to avoid thread-order nondeterminism, pinned library versions recorded (Python 3.14.6, scikit-learn 1.9.0, xgboost 3.4.0, pandas 3.0.5), and proof by execution: two independent runs produced byte-identical result files (SHA-256 verified).

Q: What are the limitations / what would you improve?

A: Three things: (1) calibration - probabilities should be checked with Brier score / reliability curves before any decision-making use, since XGB probabilities aren't guaranteed calibrated; (2) rolling-origin cross-validation across several seasons instead of a single holdout to quantify variance across eras; (3) explainability - SHAP values per state to show which factors drive swings, plus richer context features (pitch, toss, player availability).

Q: How is this different from predicting the match winner before the game?

A: Different problem, different difficulty. Pre-match prediction only uses historical form and conditions (~coin-flip territory in this repo, AUC ~0.50-0.59). Live win-probability conditioning on the current state (score, wickets, balls left, required rate) is far more informative - which is exactly why broadcasters show these curves. Conflating the two on a resume is the classic mistake this project avoids.